

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Дисциплина «Структуры и алгоритмы обработки данных»

Отчет по лабораторной работе №1

Выполнила:

Студент группы БВТ2504
Мясникова Е.А.

Проверила: Шедова А.Р

Москва, 2026 г.

Содержание

Цель работы:.....	2
Ход выполнения работы:.....	2
1. Подготовка:.....	2
2. Реализованные алгоритмы:.....	2
3. Поиск максимального элемента массива:.....	3
4. Подсчёт пар равных элементов:.....	4
5. Бинарное возведение в степень:.....	5
6. Проверка корректности алгоритмов:.....	7
7. Анализ графиков.....	8
Вывод.....	9

Цель работы: Научиться определять временную сложность алгоритмов аналитически, измерять время их выполнения и сравнивать экспериментальные результаты с теоретическими оценками.

Ход выполнения работы:

1. Подготовка:

Использовался Python 3.13.5. Данные сгенерированы для варианта 16 с seed = 46. Для каждого замера выполнялся прогрев, затем пять повторов; итоговым результатом являлась медиана времени выполнения. Перед выполнением лабораторной работы были установлены необходимые зависимости и сгенерированы данные варианта 16 командой `python scripts/generate_data.py --variant 16 --only arrays`.

2. Реализованные алгоритмы:

Была реализована функция `array_sum`, вычисляющая сумму элементов массива без использования встроенной функции `sum()`. Переменная `total` изначально равна нулю, после чего к ней последовательно прибавляется каждый элемент массива.

```
01
02 def array_sum(a: list[int]) -> int: 5 usages  8Lena
03     """Сумма элементов массива. Ожидаемая сложность: O(n) (обосновать в отчёте)."""
04     total = 0
05
06     for number in a:
07         total += number
08
09     return total
10
```

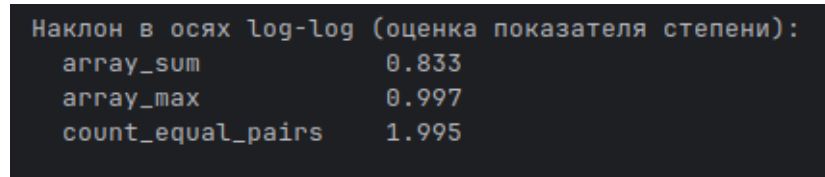
Рисунок 1 - Реализация функции вычисления суммы элементов массива

Функция проверена на массивах `[1, 2, 3]` и `[]`. Получены правильные результаты 6 и 0. Поскольку алгоритм перебирает все n элементов, его временная сложность составляет $O(n)$, а пространственная $O(1)$. После проверки были проведены замеры времени на массивах разных размеров. Для каждого размера выполнялись прогрев и пять повторов с вычислением медианы.

```
array_sum:
n= 1000    t=0.000022 c
n= 3000    t=0.000060 c
n= 10000   t=0.000200 c
n= 30000   t=0.000442 c
n= 100000  t=0.000994 c
```

Рисунок 2 - Результаты замеров функции `array_sum`

Функция всегда просматривает все n элементов, поэтому её временная сложность равна $O(n)$. Наклон графика составил 0,833, что достаточно близко к ожидаемому значению 1. Небольшое отличие связано с погрешностью измерений и работой интерпретатора Python.



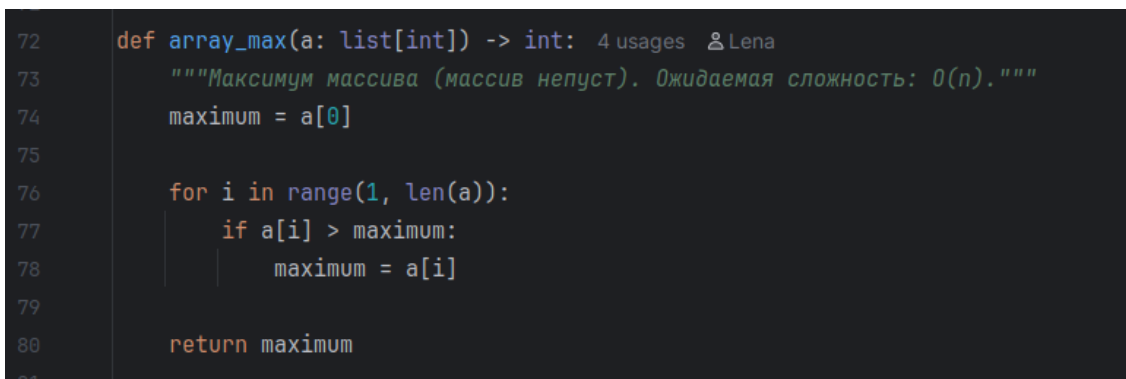
Наклон в осях log-log (оценка показателя степени):	
array_sum	0.833
array_max	0.997
count_equal_pairs	1.995

Рисунок 3 - Экспериментальная оценка наклона графика

Микровывод: функция работает правильно, результаты замеров подтверждают её линейную сложность $O(n)$.

3. Поиск максимального элемента массива:

Была реализована функция `array_max`, которая находит максимальный элемент без использования встроенной функции `max()`. Сначала за максимум принимается первый элемент массива, затем он сравнивается со всеми остальными элементами.



```
72 def array_max(a: list[int]) -> int: 4 usages 2 Lena
73     """Максимум массива (массив непуст). Ожидаемая сложность: O(n)."""
74     maximum = a[0]
75
76     for i in range(1, len(a)):
77         if a[i] > maximum:
78             maximum = a[i]
79
80     return maximum
81
```

Рисунок 4 - Реализация функции `array_max`

Функция проверена на массивах `[3, 1, 7, 2]` и `[-5, -2, -9]`. Получены правильные результаты 7 и -2. Алгоритм выполняет $n - 1$ сравнений, поэтому его временная сложность составляет $O(n)$, а пространственная - $O(1)$.

После проверки были проведены замеры времени.

```

array_max:
n= 1000 t=0.000018 c
n= 3000 t=0.000055 c
n= 10000 t=0.000175 c
n= 30000 t=0.000530 c
n= 100000 t=0.001790 c

```

Рисунок 5 - Результаты замеров функции array_max

```

Наклон в осях log-log (оценка показателя степени):
array_sum      0.833
array_max      0.997
count_equal_pairs 1.995

```

Рисунок 6 - Экспериментальная оценка наклона графика

Наклон графика для array_max составил 0,997, что практически совпадает с теоретическим значением 1.

Микровывод: функция правильно находит максимальный элемент массива, результаты измерений подтверждают линейную сложность $O(n)$.

4. Подсчёт пар равных элементов:

Была реализована функция count_equal_pairs, которая подсчитывает пары одинаковых элементов с разными индексами. Два вложенных цикла перебирают все пары (i, j) , где $i < j$. При совпадении элементов счётчик увеличивается на единицу.

```

82
83 def count_equal_pairs(a: list[int]) -> int: 5 usages 2 Lena
84     """Число пар (i, j), i < j, таких что a[i] == a[j]. Ожидаемая сложность: O(n^2)."""
85     count = 0
86
87     for i in range(len(a)):
88         for j in range(i + 1, len(a)):
89             if a[i] == a[j]:
90                 count += 1
91
92     return count
93

```

Рисунок 7 - Реализация функции count_equal_pairs

Функция проверена на пустом массиве, массиве из различных элементов и массиве [5, 5, 5]. Получены правильные результаты 0, 0 и 3. Количество сравнений равно $n(n - 1)/2$, поэтому временная сложность - $O(n^2)$, пространственная - $O(1)$.

После проверки были проведены замеры времени.

```
count_equal_pairs:
n=    500  t=0.002594 c
n=   1000  t=0.010375 c
n=   3000  t=0.087061 c
n=  10000  t=0.977861 c
n=  30000  t=9.360824 c
```

Рисунок 8 - Результаты замеров функции count_equal_pairs

```
Наклон в осях log-log (оценка показателя степени):
array_sum          0.833
array_max          0.997
count_equal_pairs  1.995
```

Рисунок 9 - Экспериментальная оценка наклона графика

Наклон графика составил 1,995, что практически совпадает с теоретическим значением 2.

Микровывод: функция правильно подсчитывает равные пары. Резкий рост времени и наклон 1,995 подтверждают квадратичную сложность $O(n^2)$.

5. Бинарное возведение в степень:

Была реализована функция `binary_row`. На каждой итерации показатель степени делится на два, а основание возводится в квадрат. Также поддерживается вычисление по модулю.

```

95 def binary_pow(x: int, n: int, mod: int | None = None) -> int: 6 usages 8 Lena
96     """Бинарное возведение в степень, n >= 0. Ожидаемая сложность: O(log n).
97
98     При заданном mod все умножения выполняются по модулю (результат x**n % mod).
99     """
100     result = 1
101
102     if mod is not None:
103         x %= mod
104         result %= mod
105
106     while n > 0:
107         if n % 2 == 1:
108             result *= x
109
110             if mod is not None:
111                 result %= mod
112
113         x *= x
114
115         if mod is not None:
116             x %= mod
117
118         n //= 2
119
120     return result
121

```

Рисунок 10 - Реализация функции `binary_pow`

Функция проверена на примерах: $2^{10}=1024$, $2^0=1$ и $2^{10} \% 1000=24$. Поскольку показатель каждый раз уменьшается вдвое, временная сложность равна $O(\log_2 n)$, пространственная - $O(1)$.

После проверки были проведены замеры по 20000 вызовов на каждую точку с модулем 1000000007.

```

binary_pow (по 20000 вызовов на точку, по модулю 1000000007):
n=      1000    log2(n)= 10.0    t=0.000000805 c
n=     10000    log2(n)= 13.3    t=0.000001049 c
n=    100000    log2(n)= 16.6    t=0.000001255 c
n=  1000000    log2(n)= 19.9    t=0.000001475 c
n= 10000000    log2(n)= 23.3    t=0.000001777 c

```

Рисунок 11 - Результаты замеров функции `binary_pow`

Время увеличивается медленно вместе с $\log_2 n$, что подтверждает логарифмическую сложность. Вычисление по модулю не позволяет числам становиться слишком большими, чтобы длинная арифметика Python не искажала результаты.

Микровывод: функция работает корректно, а результаты замеров подтверждают сложность $O(\log_2 n)$.

6. Проверка корректности алгоритмов:

Для проверки функций были добавлены тесты с помощью оператора `assert`. Использовались массивы с отрицательными числами, одним элементом, различными и повторяющимися значениями. Результаты также сравнивались со встроенными функциями `sum()`, `max()` и `pow()` на случайных данных.

```
105     # Собственные проверки граничных и типовых случаев
106     assert array_sum([-5, 0, 5]) == 0
107     assert array_max([-7]) == -7
108     assert count_equal_pairs([1, 2, 3]) == 0
109     assert count_equal_pairs([1, 2, 1, 2]) == 2
110     assert binary_pow(x: 5, n: 1) == 5
111
112     print("self_check: OK")
113
```

Рисунок 12 - Собственные проверки граничных и типовых случаев

После запуска функции `self_check` все проверки были успешно выполнены. Сообщение `self_check: OK` означает, что ошибок и несовпадений не обнаружено.

```
(.venv) elena@tuf:~/data
self_check: OK
```

Рисунок 13 - Результат выполнения проверок

Микровывод: все реализованные алгоритмы прошли граничные, типовые и случайные тесты, поэтому их реализация считается корректной.

7. Анализ графиков

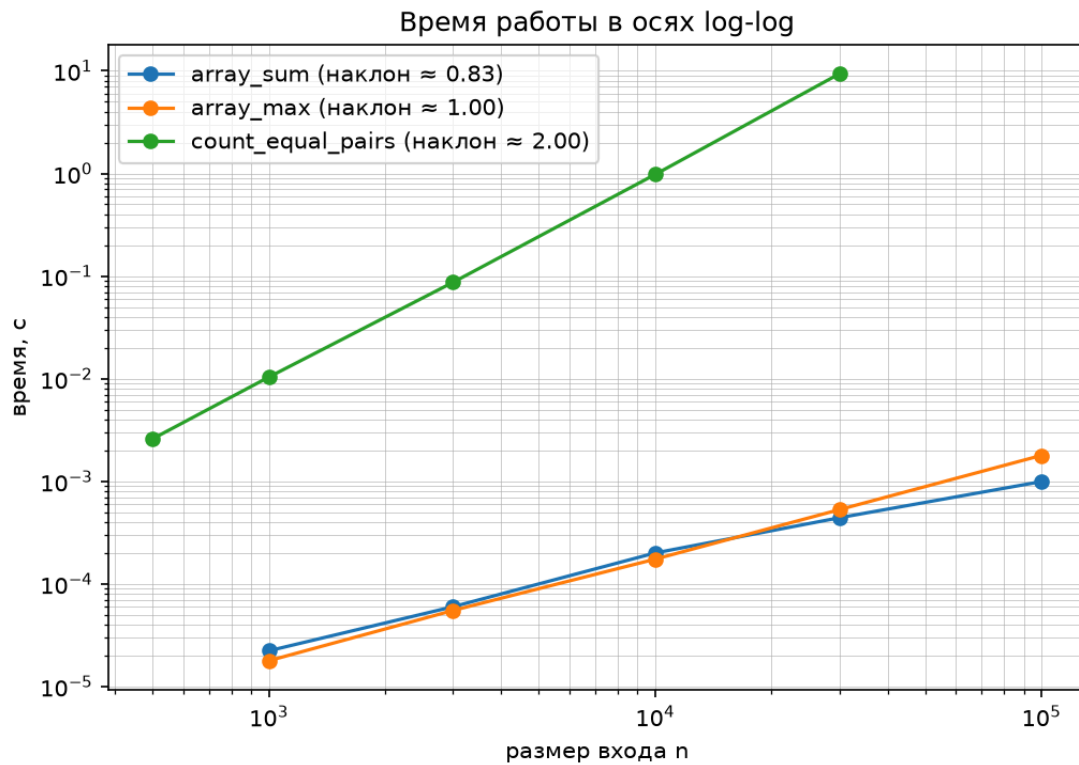


Рисунок 14 - Зависимость времени работы алгоритмов от размера массива

На графике `array_sum` и `array_max` показывают приблизительно линейный рост. Их наклоны равны 0,833 и 0,997 при теоретическом значении 1. Наклон `count_equal_pairs` равен 1,995, что практически совпадает с теоретическим значением 2 для сложности $O(n^2)$.

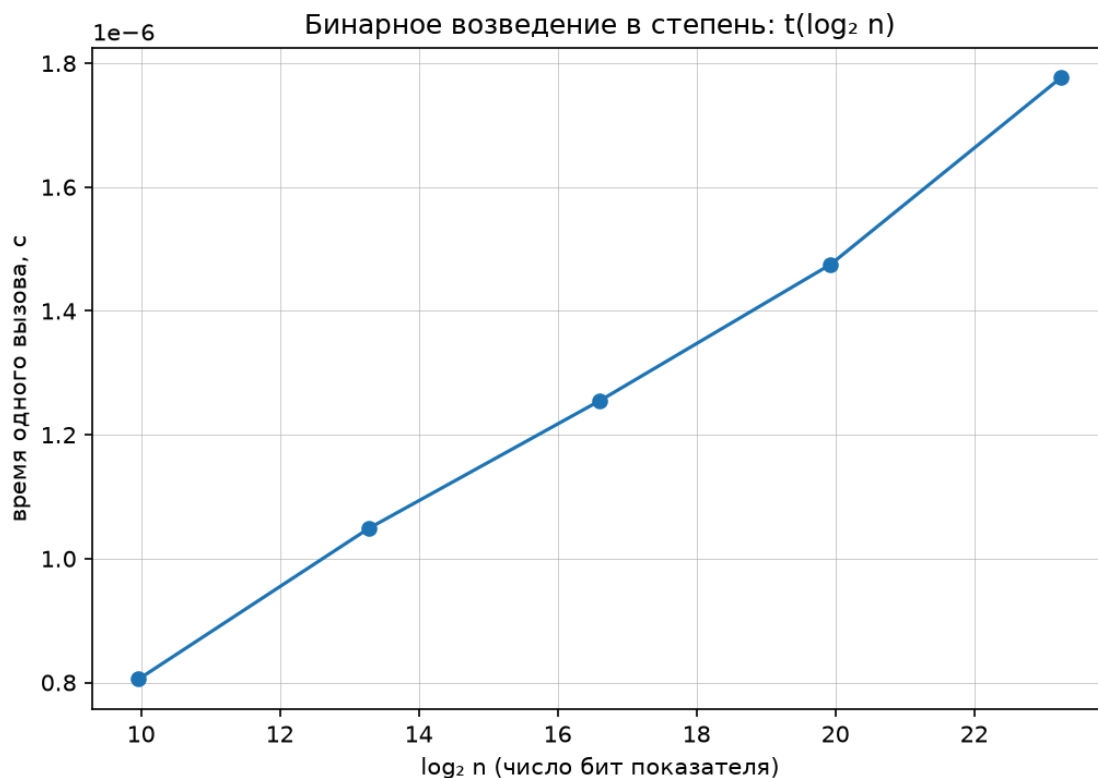


Рисунок 15 - Зависимость времени бинарного возведения в степень от $\log_2(n)$

Для `binary_pow` выбраны линейные оси по $\log_2(n)$, поскольку время алгоритма прямо зависит от количества делений показателя на два. На графике видно, что время увеличивается медленно, подтверждая сложность $O(\log_2 n)$.

Микровывод: экспериментальные графики соответствуют теоретическим оценкам сложности реализованных алгоритмов.

Вывод

В ходе лабораторной работы были реализованы и проверены четыре алгоритма: сумма элементов массива, поиск максимума, подсчёт равных пар и бинарное возведение в степень. Теоретические оценки сложности $O(n)$, $O(n^2)$ и $O(\log_2 n)$ подтвердились результатами измерений и графиками. Небольшие расхождения связаны с погрешностью замеров и особенностями работы Python. Таким образом, цель работы достигнута.